```
import sys

print("Python Version:", sys.version)
print("Project Started Successfully!")
```

```
Python Version: 3.13.18 (main, Oct 26 2026, 15:28:15) [GCC 13.3.0]
Project Started Successfully!
```

```
!pip install pandas numpy scikit-learn matplotlib seaborn
!pip install fastapi uvicorn joblib
!pip install prometheus-client prometheus-fastapi-instrumentator
!pip install evidently
```

```
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx>=0.22->lite
Requirement already satisfied: joblib in /usr/local/lib/python3.13/dist-packages (from nltk>=3.6.7->evidently) (1.6.0)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.13/dist-packages (from nltk>=3.6.7->evidently) (2
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-packages (from nltk>=3.6.7->evidently) (4.67.3)
Requirement already satisfied: protobuf<8.0,>=5.0 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-proto>=1.
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.3.5->pand
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.3.5->pandas[parquet
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.3.5->pandas[parqu
Requirement already satisfied: pyarrow>=10.0.1 in /usr/local/lib/python3.13/dist-packages (from pandas[parquet]>=1.3.5->ev
Collecting faker>=5.0.0 (from polyfactory>=2.6.3->litestar>=2.19.0->evidently)
  Downloading faker-40.44.0-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic>=1.10.16->
Requirement already satisfied: pydantic-core==2.50.1 in /usr/local/lib/python3.13/dist-packages (from pydantic>=1.10.16->e
Requirement already satisfied: typing-inspection>=0.4.4 in /usr/local/lib/python3.13/dist-packages (from pydantic>=1.10.16
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas>=1
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests>=2.32.0-
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.13/dist-packages (from rich>=13->evidently)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from rich>=13->evidentl
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.13/dist-packages (from markdown-it-py>=2.2.0->rich>=13
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.1.1->
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib->nltk>=3.6.7->evid
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.13/dist-packages (from statsmodels>=0.14.0->evidentl
Requirement already satisfied: formulaic>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from statsmodels>=0.14.0->evid
Requirement already satisfied: interface-meta>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->st
Requirement already satisfied: narwhals>=1.17 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmode
Requirement already satisfied: wrapt>=1.17.0rc1 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmo
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from typer>=0.3->evidently)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-packages (from typer>=0.3->evidently
Collecting mypy-extensions>=0.3.0 (from typing-inspect>=0.9.0->evidently)
  Downloading mypy_extensions-1.2.0-py3-none-any.whl.metadata (1.1 kB)
Requirement already satisfied: httptools>=0.8.0 in /usr/local/lib/python3.13/dist-packages (from uvicorn[standard]>=0.22.0
Requirement already satisfied: python-dotenv>=0.13 in /usr/local/lib/python3.13/dist-packages (from uvicorn[standard]>=0.2
Requirement already satisfied: uvloop>=0.15.1 in /usr/local/lib/python3.13/dist-packages (from uvicorn[standard]>=0.22.0->
Requirement already satisfied: watchfiles>=0.20 in /usr/local/lib/python3.13/dist-packages (from uvicorn[standard]>=0.22.0
Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.13/dist-packages (from uvicorn[standard]>=0.22.0
Downloading evidently-0.7.23-py3-none-any.whl (11.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.7/11.7 MB 94.5 MB/s  0:00:00
Downloading dynaconf-3.3.5-py3-none-any.whl (271 kB)
Downloading iterative_telemetry-0.0.10-py3-none-any.whl (10 kB)
Downloading litestar-2.25.0-py3-none-any.whl (584 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 584.7/584.7 kB 18.8 MB/s  0:00:00
Downloading litestar_htmx-0.5.0-py3-none-any.whl (10.0 kB)
Downloading msgspec-0.22.0-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (230 kB)
Downloading multipart-2.0.1-py3-none-any.whl (16 kB)
Downloading opentelemetry_proto-1.45.2-py3-none-any.whl (72 kB)
Downloading polyfactory-3.5.0-py3-none-any.whl (63 kB)
Downloading faker-40.44.0-py3-none-any.whl (2.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 51.6 MB/s  0:00:00
Downloading typing_inspect-0.9.0-py3-none-any.whl (8.8 kB)
Downloading mypy_extensions-1.2.0-py3-none-any.whl (5.1 kB)
Downloading ujson-6.1.0-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (56 kB)
Downloading uuid6-2025.0.1-py3-none-any.whl (7.0 kB)
Downloading appdirs-1.4.4-py2.py3-none-any.whl (9.6 kB)
Downloading rich_click-1.9.9-py3-none-any.whl (71 kB)
Installing collected packages: appdirs, uuid6, ujson, opentelemetry-proto, mypy-extensions, multipart, msgspec, litestar-h
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16/16 [evidently]
Successfully installed appdirs-1.4.4 dynaconf-3.3.5 evidently-0.7.23 faker-40.44.0 iterative-telemetry-0.0.10 litestar-2.2
```

```
import pandas as pd

reference = pd.read_csv("/content/Section7_Q3_Online_Payment_Fraud_Reference.csv")
current = pd.read_csv("/content/Section7_Q3_Payment_Fraud_Current_Drift.csv")

print("Reference Dataset:", reference.shape)
print("Current Dataset:", current.shape)

display(reference.head())
display(current.head())
```

```
Reference Dataset: (1100, 14)
Current Dataset: (350, 14)
```

	Transaction_ID	Transaction_Amount_INR	Transaction_Hour	Payment_Method	Transaction_Channel	Device_Status	Location_Ty
0	TXN000001	1145.93	8	UPI	POS	Known	Domes
1	TXN000002	581.26	16	UPI	POS	Known	Domes
2	TXN000003	NaN	21	UPI	Web	Known	Domes
3	TXN000004	626.13	1	NetBanking	POS	Known	Domes
4	TXN000005	1125.22	18	UPI	Mobile App	Known	Domes

	Transaction_ID	Transaction_Amount_INR	Transaction_Hour	Payment_Method	Transaction_Channel	Device_Status	Location_Ty
0	TXN001101	610.87	3	UPI	POS	New	Internatio
1	TXN001102	198.48	4	UPI	Mobile App	New	Domes
2	TXN001103	1227.60	17	UPI	Mobile App	Known	Domes
3	TXN001104	1331.92	1	Card	POS	Known	Internatio
4	TXN001105	1055.85	15	NaN	POS	Known	Internatio

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
reference = pd.read_csv("/content/Section7_Q3_Online_Payment_Fraud_Reference.csv")
current = pd.read_csv("/content/Section7_Q3_Payment_Fraud_Current_Drift.csv")

print("Reference Dataset Shape:", reference.shape)
print("Current Dataset Shape:", current.shape)

display(reference.head())
```

```
Reference Dataset Shape: (1100, 14)
Current Dataset Shape: (350, 14)
```

	Transaction_ID	Transaction_Amount_INR	Transaction_Hour	Payment_Method	Transaction_Channel	Device_Status	Location_Ty
0	TXN000001	1145.93	8	UPI	POS	Known	Domes
1	TXN000002	581.26	16	UPI	POS	Known	Domes
2	TXN000003	NaN	21	UPI	Web	Known	Domes
3	TXN000004	626.13	1	NetBanking	POS	Known	Domes
4	TXN000005	1125.22	18	UPI	Mobile App	Known	Domes

```
print("Missing values in Reference Dataset:")
print(reference.isnull().sum())

print("\nMissing values in Current Dataset:")
print(current.isnull().sum())
```

```
Missing values in Reference Dataset:
Transaction_ID                    0
Transaction_Amount_INR           26
Transaction_Hour                  0
Payment_Method                   26
Transaction_Channel               0
Device_Status                    26
Location_Type                     0
Transactions_Last_Hour           26
Account_Age_Days                  0
Fraud_Risk_Signal                26
Failed_OTP_Attempts               0
Previous_Disputes                 0
Minutes_Since_Last_Transaction    0
Fraud_Label                       0
```

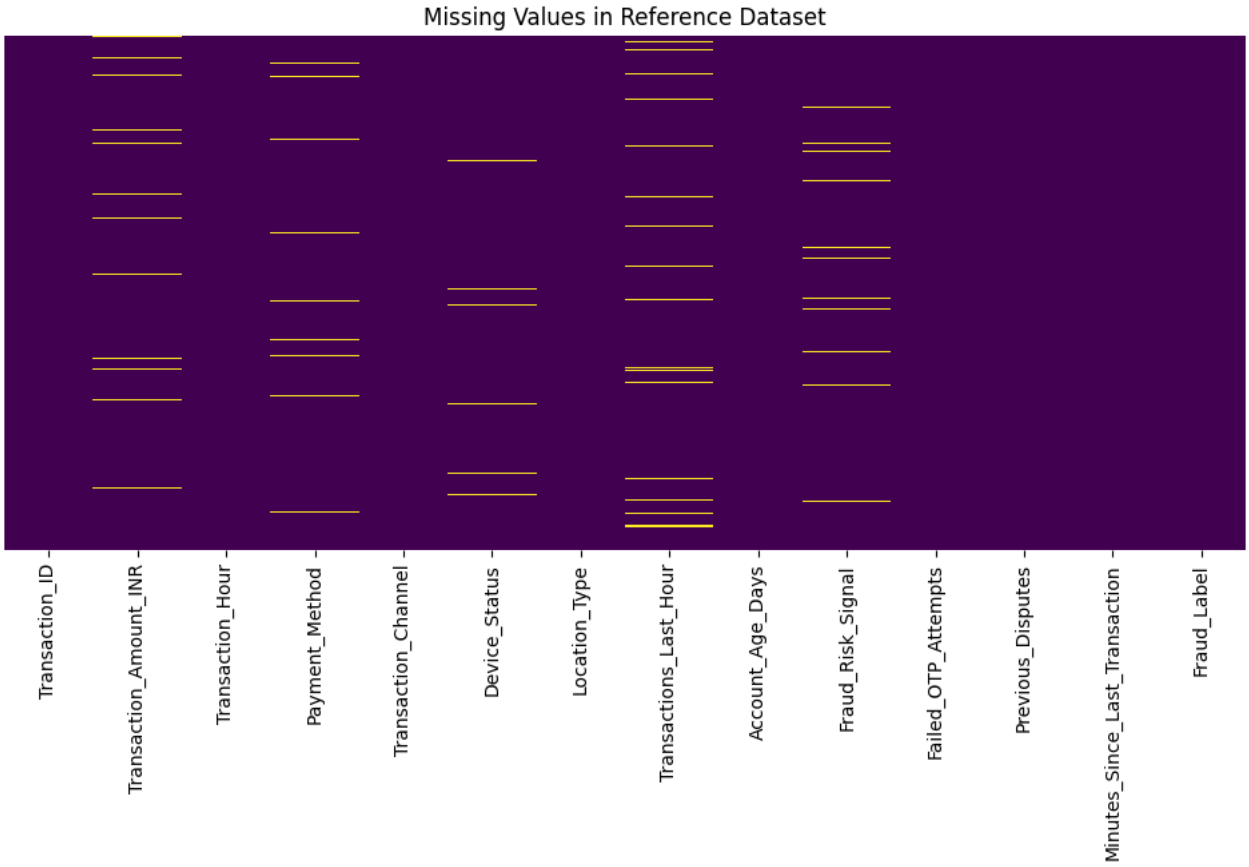
```
dtype: int64

Missing values in Current Dataset:
Transaction_ID                    0
Transaction_Amount_INR            8
Transaction_Hour                  0
Payment_Method                    8
Transaction_Channel               0
Device_Status                     8
Location_Type                     0
Transactions_Last_Hour            8
Account_Age_Days                  0
Fraud_Risk_Signal                 8
Failed_OTP_Attempts               0
Previous_Disputes                 0
Minutes_Since_Last_Transaction    0
Fraud_Label                       0
dtype: int64
```

```
plt.figure(figsize=(12,5))

sns.heatmap(
    reference.isnull(),
    cbar=False,
    yticklabels=False,
    cmap="viridis"
)

plt.title("Missing Values in Reference Dataset")
plt.show()
```




```
reference["Transaction_Amount_INR"] = (
    reference["Transaction_Amount_INR"].replace(-999, np.nan)
)

current["Transaction_Amount_INR"] = (
    current["Transaction_Amount_INR"].replace(-999, np.nan)
)

print("Invalid values handled successfully!")
```

```
Invalid values handled successfully!
```

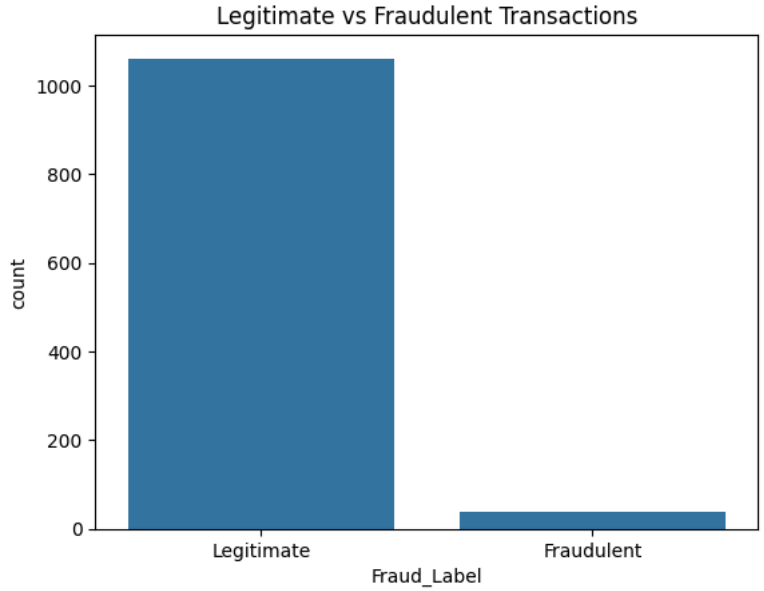
```
print(reference["Fraud_Label"].value_counts())

sns.countplot(
    data=reference,
```

```
    x="Fraud_Label"
)

plt.title("Legitimate vs Fraudulent Transactions")
plt.show()
```

```
Fraud_Label
Legitimate    1061
Fraudulent      39
Name: count, dtype: int64
```




```
reference["Hour_Sin"] = np.sin(
    2 * np.pi * reference["Transaction_Hour"] / 24
)

reference["Hour_Cos"] = np.cos(
    2 * np.pi * reference["Transaction_Hour"] / 24
)

reference["High_Amount"] = (
    reference["Transaction_Amount_INR"] > 2000
).astype(int)

print("Feature Engineering Completed!")

display(reference.head())
```

```
Feature Engineering Completed!
```

	Transaction_ID	Transaction_Amount_INR	Transaction_Hour	Payment_Method	Transaction_Channel	Device_Status	Location_Ty
0	TXN000001	1145.93	8	UPI	POS	Known	Domes
1	TXN000002	581.26	16	UPI	POS	Known	Domes
2	TXN000003	NaN	21	UPI	Web	Known	Domes
3	TXN000004	626.13	1	NetBanking	POS	Known	Domes
4	TXN000005	1125.22	18	UPI	Mobile App	Known	Domes

```
X = reference.drop(
    columns=["Transaction_ID", "Fraud_Label"]
)

y = reference["Fraud_Label"].map({
    "Legitimate": 0,
    "Fraudulent": 1
})

print("Input Features:", X.shape)
print("Output Labels:", y.shape)
```

```
print("\nTarget Distribution:")
print(y.value_counts())
```

```
Input Features: (1100, 15)
Output Labels: (1100,)

Target Distribution:
Fraud_Label
0    1061
1      39
Name: count, dtype: int64
```

```
numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

categorical_features = X.select_dtypes(
    include=["object"]
).columns.tolist()

print("Numerical Features:")
print(numeric_features)

print("\nCategorical Features:")
print(categorical_features)
```

```
Numerical Features:
['Transaction_Amount_INR', 'Transaction_Hour', 'Transactions_Last_Hour', 'Account_Age_Days', 'Fraud_Risk_Signal', 'Failed_OT

Categorical Features:
['Payment_Method', 'Transaction_Channel', 'Device_Status', 'Location_Type']
```

```
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median"))
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

print("Preprocessing Pipeline Created!")
```

```
Preprocessing Pipeline Created!
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (880, 15)
Testing Data: (220, 15)
```

```
from sklearn.ensemble import RandomForestClassifier

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
        n_estimators=200,
        class_weight="balanced",
        random_state=42
    ))
])

model.fit(X_train, y_train)

print("Random Forest Model Trained Successfully!")
```

```
Random Forest Model Trained Successfully!
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_pred, zero_division=0))

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=["Legitimate", "Fraudulent"],
    zero_division=0
))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.9636363636363636
Precision: 0.0
Recall: 0.0
F1 Score: 0.0

Classification Report:
              precision    recall  f1-score   support

  Legitimate       0.96      1.00      0.98       212
  Fraudulent       0.00      0.00      0.00         8

    accuracy                           0.96       220
   macro avg       0.48      0.50      0.49       220
weighted avg       0.93      0.96      0.95       220


Confusion Matrix:
[[212   0]
 [  8   0]]
```

```
import joblib

joblib.dump(model, "/content/fraud_model.pkl")

print("Model Saved Successfully!")
```

```
Model Saved Successfully!
```

```
!pip install fastapi uvicorn joblib prometheus-client prometheus-fastapi-instrumentator
```

```
Requirement already satisfied: fastapi in /usr/local/lib/python3.13/dist-packages (0.141.1)
Requirement already satisfied: uvicorn in /usr/local/lib/python3.13/dist-packages (0.54.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.13/dist-packages (1.6.0)
Requirement already satisfied: prometheus-client in /usr/local/lib/python3.13/dist-packages (0.26.0)
Requirement already satisfied: prometheus-fastapi-instrumentator in /usr/local/lib/python3.13/dist-packages (8.1.0)
Requirement already satisfied: starlette>=0.46.0 in /usr/local/lib/python3.13/dist-packages (from fastapi) (1.10.0)
Requirement already satisfied: pydantic>=2.9.0 in /usr/local/lib/python3.13/dist-packages (from fastapi) (2.14.1)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.13/dist-packages (from fastapi) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from fastapi) (0.4.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-packages (from fastapi) (0.0.5)
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.13/dist-packages (from uvicorn) (8.5.0)
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.13/dist-packages (from uvicorn) (0.16.0)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib) (3.1.2)
Requirement already satisfied: anyio<5,>=4.0.0 in /usr/local/lib/python3.13/dist-packages (from starlette>=0.46.0->fastapi)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.13/dist-packages (from anyio<5,>=4.0.0->starlette>=0.46.0
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic>=2.9.0->fast
Requirement already satisfied: pydantic-core==2.50.1 in /usr/local/lib/python3.13/dist-packages (from pydantic>=2.9.0->fasta
```

```
import os

print(os.path.exists("/content/fraud_model.pkl"))
```

```
True
```

```
%%writefile main.py
```

```
from fastapi import FastAPI, HTTPException, Request
from pydantic import BaseModel, Field
import pandas as pd
import numpy as np
import joblib
import logging
import time

from prometheus_client import Counter, Histogram
from prometheus_fastapi_instrumentator import Instrumentator

# Initialize FastAPI
app = FastAPI(
    title="Online Payment Fraud Detection System",
    description="Machine Learning Based Fraud Detection API",
    version="1.0"
)

# Load trained model
model = joblib.load("/content/fraud_model.pkl")

# Configure logging
logging.basicConfig(
    filename="/content/fraud_app.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

# Prometheus metrics
REQUEST_COUNT = Counter(
    "transaction_request_count",
    "Total transaction requests"
)

PREDICTION_COUNT = Counter(
    "fraud_prediction_count",
    "Total predictions"
)

ERROR_COUNT = Counter(
    "application_error_count",
    "Total errors"
)

RESPONSE_TIME = Histogram(
    "transaction_response_time_seconds",
    "Transaction processing time"
)

# Prometheus instrumentation
Instrumentator().instrument(app).expose(
    app,
    endpoint="/metrics"
)

# Transaction input format
class Transaction(BaseModel):

    Transaction_Amount_INR: float
    Transaction_Hour: int = Field(ge=0, le=23)

    Payment_Method: str
    Transaction_Channel: str
    Device_Status: str
    Location_Type: str

    Transactions_Last_Hour: float
    Account_Age_Days: int
    Fraud_Risk_Signal: float
    Failed_OTP_Attempts: int
    Previous_Disputes: int
    Minutes_Since_Last_Transaction: int


# Health endpoint
@app.get("/health")
def health():

    return {
        "status": "healthy",
        "model": "loaded"
    }
```

```
# Prediction endpoint
@app.post("/predict")
def predict(transaction: Transaction):

    start_time = time.time()
    REQUEST_COUNT.inc()

    try:

        data = transaction.model_dump()

        # Handle invalid transaction amount
        if data["Transaction_Amount_INR"] == -999:
            data["Transaction_Amount_INR"] = np.nan

        # Feature engineering
        data["Hour_Sin"] = np.sin(
            2 * np.pi * data["Transaction_Hour"] / 24
        )

        data["Hour_Cos"] = np.cos(
            2 * np.pi * data["Transaction_Hour"] / 24
        )

        data["High_Amount"] = int(
            data["Transaction_Amount_INR"] > 2000
        ) if not pd.isna(
            data["Transaction_Amount_INR"]
        ) else 0

        # Convert to DataFrame
        input_df = pd.DataFrame([data])

        # Predict
        prediction = int(model.predict(input_df)[0])

        probability = float(
            model.predict_proba(input_df)[0][1]
        )

        result = (
            "Fraudulent" if prediction == 1
            else "Legitimate"
        )

        PREDICTION_COUNT.inc()

        processing_time = time.time() - start_time

        RESPONSE_TIME.observe(processing_time)

        # Record request and result
        logging.info(
            f"Transaction={data}, "
            f"Prediction={result}, "
            f"Fraud_Probability={probability:.4f}, "
            f"Processing_Time={processing_time:.4f}s"
        )

        return {
            "prediction": result,
            "fraud_probability": probability,
            "processing_time": processing_time
        }

    except Exception as e:

        ERROR_COUNT.inc()

        logging.exception("Prediction error")

        raise HTTPException(
            status_code=500,
            detail="Prediction failed"
        )
```

```
Writing main.py
```

```
!nohup uvicorn main:app --host 0.0.0.0 --port 8000 > api.log 2>&1 &
```

```
from google.colab.output import eval_js

url = eval_js("google.colab.kernel.proxyPort(8000)")

print("FastAPI URL:", url)
print("Swagger URL:", url + "/docs")
```

```
FastAPI URL: https://8000-m-s-euw4b3-3q2epebgdduab-b.europe-west4.prod.r.colab.dev
Swagger URL: https://8000-m-s-euw4b3-3q2epebgdduab-b.europe-west4.prod.r.colab.dev/docs
```

```
!cat /content/api.log
```

```
INFO:     Started server process [5326]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:43636 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:43648 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:57622 - "GET / HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:57628 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```

```
!grep -n "health" /content/main.py
```

```
77:@app.get("/health")
78:def health():
81:        "status": "healthy",
```

```
import requests

response = requests.get("http://127.0.0.1:8000/health")

print("Status:", response.status_code)
print("Response:", response.text)
```

```
Status: 200
Response: {"status":"healthy","model":"loaded"}
```

```
response = requests.get("http://127.0.0.1:8000/openapi.json")

print("Status:", response.status_code)

if response.status_code == 200:
    data = response.json()
    print("Available endpoints:")
    for path in data["paths"]:
        print(path)
```

```
Status: 200
Available endpoints:
/metrics
/health
/predict
```

```
!grep -n -E "FastAPI|@app.get|@app.post|health" /content/main.py
```

```
2:from fastapi import FastAPI, HTTPException, Request
13:# Initialize FastAPI
14:app = FastAPI(
77:@app.get("/health")
78:def health():
81:        "status": "healthy",
87:@app.post("/predict")
```

```
!tail -20 /content/api.log
```

```
INFO:     Started server process [5326]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:43636 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:43648 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:57622 - "GET / HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:57628 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:55288 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:57738 - "GET /openapi.json HTTP/1.1" 200 OK
```

```
import requests

url = "http://127.0.0.1:8000/predict"

transaction = {
    "Transaction_Amount_INR": 5000,
```

```
    "Transaction_Hour": 23,
    "Payment_Method": "Card",
    "Transaction_Channel": "Online",
    "Device_Status": "New",
    "Location_Type": "International",
    "Transactions_Last_Hour": 8,
    "Account_Age_Days": 10,
    "Fraud_Risk_Signal": 0.9,
    "Failed_OTP_Attempts": 3,
    "Previous_Disputes": 2,
    "Minutes_Since_Last_Transaction": 2
}

response = requests.post(url, json=transaction)

print("Status:", response.status_code)
print("Result:", response.json())
```

```
Status: 200
Result: {'prediction': 'Legitimate', 'fraud_probability': 0.13, 'processing_time': 0.15944766998291016}
```

```
response = requests.get("http://127.0.0.1:8000/metrics")

print(response.text[:3000])
```

```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 1122.0
python_gc_objects_collected_total{generation="1"} 35.0
python_gc_objects_collected_total{generation="2"} 48.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 138.0
python_gc_collections_total{generation="1"} 12.0
python_gc_collections_total{generation="2"} 1.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="13",patchlevel="18",version="3.13.18"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 8.4310016e+08
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 2.34610688e+08
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.79397703639e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 5.17
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 17.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP transaction_request_count_total Total transaction requests
# TYPE transaction_request_count_total counter
transaction_request_count_total 1.0
# HELP transaction_request_count_created Total transaction requests
# TYPE transaction_request_count_created gauge
transaction_request_count_created 1.793977040673813e+09
# HELP fraud_prediction_count_total Total predictions
# TYPE fraud_prediction_count_total counter
fraud_prediction_count_total 1.0
# HELP fraud_prediction_count_created Total predictions
# TYPE fraud_prediction_count_created gauge
fraud_prediction_count_created 1.7939770406738687e+09
# HELP application_error_count_total Total errors
# TYPE application_error_count_total counter
application_error_count_total 0.0
# HELP application_error_count_created Total errors
# TYPE application_error_count_created gauge
application_error_count_created 1.7939770406738918e+09
# HELP transaction_response_time_seconds Transaction processing time
# TYPE transaction_response_time_seconds histogram
transaction_response_time_seconds_bucket{le="0.005"} 0.0
transaction_response_time_seconds_bu
```

```
import pandas as pd
import numpy as np
```

```
# Load both datasets
reference = pd.read_csv("/content/Section7_Q3_Online_Payment_Fraud_Reference.csv")
current = pd.read_csv("/content/Section7_Q3_Payment_Fraud_Current_Drift.csv")

# Replace invalid amount values
reference["Transaction_Amount_INR"] = (
    reference["Transaction_Amount_INR"].replace(-999, np.nan)
)

current["Transaction_Amount_INR"] = (
    current["Transaction_Amount_INR"].replace(-999, np.nan)
)

# Remove transaction ID and target label for feature drift analysis
reference_features = reference.drop(
    columns=["Transaction_ID", "Fraud_Label"]
)

current_features = current.drop(
    columns=["Transaction_ID", "Fraud_Label"]
)

print("Reference shape:", reference_features.shape)
print("Current shape:", current_features.shape)
```

```
Reference shape: (1100, 12)
Current shape: (350, 12)
```

```
!pip install -q evidently
```

```
from evidently import Report
from evidently.presets import DataDriftPreset

# Create drift report
report = Report([
    DataDriftPreset()
])

# Compare reference and current data
snapshot = report.run(
    reference_data=reference_features,
    current_data=current_features
)

# Save HTML report
snapshot.save_html("/content/data_drift_report.html")

print("Data drift report generated successfully!")
```

```
Data drift report generated successfully!
```

```
from google.colab import files

files.download("/content/data_drift_report.html")
```

```

```

```
import joblib
import numpy as np
import pandas as pd

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

# Load the existing trained model
model = joblib.load("/content/fraud_model.pkl")

# Prepare current dataset
test_data = current.copy()

test_data["Transaction_Amount_INR"] = (
    test_data["Transaction_Amount_INR"].replace(-999, np.nan)
)
```

```
# Apply the same feature engineering
test_data["Hour_Sin"] = np.sin(
    2 * np.pi * test_data["Transaction_Hour"] / 24
)

test_data["Hour_Cos"] = np.cos(
    2 * np.pi * test_data["Transaction_Hour"] / 24
)

test_data["High_Amount"] = (
    test_data["Transaction_Amount_INR"] > 2000
).astype(int)

# Separate features and actual labels
X_current = test_data.drop(
    columns=["Transaction_ID", "Fraud_Label"]
)

y_current = test_data["Fraud_Label"].map({
    "Legitimate": 0,
    "Fraudulent": 1
})

# Make predictions
predictions = model.predict(X_current)

# Evaluate performance
print("Accuracy:", accuracy_score(y_current, predictions))
print("Precision:", precision_score(y_current, predictions, zero_division=0))
print("Recall:", recall_score(y_current, predictions, zero_division=0))
print("F1 Score:", f1_score(y_current, predictions, zero_division=0))

print("\nClassification Report:")
print(classification_report(
    y_current,
    predictions,
    target_names=["Legitimate", "Fraudulent"],
    zero_division=0
))

print("Confusion Matrix:")
print(confusion_matrix(y_current, predictions))
```

```
Accuracy: 0.9628571428571429
Precision: 0.0
Recall: 0.0
F1 Score: 0.0

Classification Report:
              precision    recall  f1-score   support

  Legitimate       0.96      1.00      0.98       337
  Fraudulent       0.00      0.00      0.00        13

    accuracy                           0.96       350
   macro avg       0.48      0.50      0.49       350
weighted avg       0.93      0.96      0.94       350

Confusion Matrix:
[[337   0]
 [ 13   0]]
```

```
import numpy as np
import pandas as pd

# Get fraud probabilities
probabilities = model.predict_proba(X_test)[:, 1]

# Check probability distribution
print("Minimum probability:", probabilities.min())
print("Maximum probability:", probabilities.max())
print("Average probability:", probabilities.mean())

# Compare actual labels with predicted probabilities
results = pd.DataFrame({
    "Actual_Label": y_test,
    "Fraud_Probability": probabilities
})

print("\nFraud probability by actual class:")
print(results.groupby("Actual_Label")["Fraud_Probability"].describe())
```

```
Minimum probability: 0.0
Maximum probability: 0.21
Average probability: 0.035090909090909096

Fraud probability by actual class:
              count      mean       std  min    25%    50%   75%   max
Actual_Label
0             212.0  0.035283  0.036414  0.0  0.010  0.025  0.05  0.21
1               8.0  0.030000  0.027386  0.0  0.005  0.025  0.05  0.07
```

```
import numpy as np
import pandas as pd

print("Model classes:", model.classes_)

print("\nTraining label distribution:")
print(pd.Series(y_train).value_counts())

print("\nTesting label distribution:")
print(pd.Series(y_test).value_counts())

probabilities = model.predict_proba(X_test)[:, 1]

results = pd.DataFrame({
    "Actual": np.asarray(y_test),
    "Fraud_Probability": probabilities
})

print("\nFraud probability for each actual class:")
print(
    results.groupby("Actual")["Fraud_Probability"]
    .agg(["min", "max", "mean", "median"])
)
```

```
Model classes: [0 1]

Training label distribution:
Fraud_Label
0    849
1     31
Name: count, dtype: int64

Testing label distribution:
Fraud_Label
0    212
1      8
Name: count, dtype: int64

Fraud probability for each actual class:
        min   max      mean  median
Actual
0       0.0  0.21  0.035283   0.025
1       0.0  0.07  0.030000   0.025
```

```
print("Model type:")
print(type(model))

print("\nModel structure:")
print(model)

print("\nTraining fraud samples:")
print(X_train[y_train == 1].head(5))

print("\nTraining legitimate samples:")
print(X_train[y_train == 0].head(5))
```

```
43             0
943            0
51             0

Training legitimate samples:
     Transaction_Amount_INR  Transaction_Hour Payment_Method  \
536                 1175.12                 3            UPI
601                  786.60                23            UPI
956                  214.27                17            UPI
170                  243.19                19         Wallet
451                  482.61                 5     NetBanking

    Transaction_Channel Device_Status Location_Type  Transactions_Last_Hour  \
536                 POS           New      Domestic                     3.0
601                 POS           New      Domestic                     2.0
956                 Web         Known      Domestic                     1.0
170                 POS         Known      Domestic                     3.0
451                 Web           New      Domestic                     2.0

     Account_Age_Days  Fraud_Risk_Signal  Failed_OTP_Attempts  \
536              2446               38.0                    1
601               426               71.0                    0
956              1376               89.0                    0
170               796               45.0                    1
451               494               33.0                    0

     Previous_Disputes  Minutes_Since_Last_Transaction  Hour_Sin  Hour_Cos  \
536                  1                             145  0.707107  0.707107
601                  0                             167 -0.258819  0.965926
956                  0                             121 -0.965926 -0.258819
170                  0                              24 -0.965926  0.258819
451                  0                             154  0.965926  0.258819

     High_Amount
536            0
601            0
956            0
170            0
451            0
```

```
!pip install -q imbalanced-learn
```

```
from imblearn.over_sampling import RandomOverSampler
from imblearn.pipeline import Pipeline as ImbPipeline

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

# Create an improved pipeline
improved_model = ImbPipeline(steps=[
    ("sampler", RandomOverSampler(
        sampling_strategy=0.5,
        random_state=42
    )),

    ("preprocessor", model.named_steps["preprocessor"]),

    ("classifier", RandomForestClassifier(
        n_estimators=300,
        class_weight=None,
        min_samples_leaf=2,
        random_state=42
    ))
])

# Train using training data only
improved_model.fit(X_train, y_train)

# Predict on the original, untouched test data
y_pred_new = improved_model.predict(X_test)

# Evaluate
print("IMPROVED MODEL RESULTS")

print("Accuracy:", accuracy_score(y_test, y_pred_new))
print("Precision:", precision_score(y_test, y_pred_new, zero_division=0))
print("Recall:", recall_score(y_test, y_pred_new, zero_division=0))
print("F1 Score:", f1_score(y_test, y_pred_new, zero_division=0))
```

```
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred_new,
    target_names=["Legitimate", "Fraudulent"],
    zero_division=0
))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_new))
```

```
IMPROVED MODEL RESULTS
Accuracy: 0.9636363636363636
Precision: 0.0
Recall: 0.0
F1 Score: 0.0

Classification Report:
              precision    recall  f1-score   support

  Legitimate       0.96      1.00      0.98       212
  Fraudulent       0.00      0.00      0.00         8

    accuracy                           0.96       220
   macro avg       0.48      0.50      0.49       220
weighted avg       0.93      0.96      0.95       220


Confusion Matrix:
[[212   0]
 [  8   0]]
```

```
import joblib

joblib.dump(improved_model, "/content/fraud_model_v2.pkl")

print("Improved model saved successfully!")
```

```
Improved model saved successfully!
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

models = {
    "Original Model": model,
    "Improved Model": improved_model
}

for name, trained_model in models.items():
    predictions = trained_model.predict(X_test)

    print("\n", name)
    print("Accuracy:", accuracy_score(y_test, predictions))
    print("Precision:", precision_score(y_test, predictions, zero_division=0))
    print("Recall:", recall_score(y_test, predictions, zero_division=0))
    print("F1 Score:", f1_score(y_test, predictions, zero_division=0))
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, predictions))
```

```
 Original Model
Accuracy: 0.9636363636363636
Precision: 0.0
Recall: 0.0
F1 Score: 0.0
Confusion Matrix:
[[212   0]
 [  8   0]]

 Improved Model
Accuracy: 0.9636363636363636
Precision: 0.0
Recall: 0.0
F1 Score: 0.0
Confusion Matrix:
[[212   0]
 [  8   0]]
```

```
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix, classification_report

# Get improved model probabilities
v2_probabilities = improved_model.predict_proba(X_test)[:, 1]
```

```
results = pd.DataFrame({
    "Actual": np.asarray(y_test),
    "Fraud_Probability": v2_probabilities
})

print("Probability summary:")
print(results.groupby("Actual")["Fraud_Probability"].describe())

# Test different thresholds
for threshold in [0.5, 0.2, 0.1, 0.05, 0.02, 0.01]:
    predictions = (v2_probabilities >= threshold).astype(int)

    print("\nThreshold:", threshold)
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, predictions))
```

```
Probability summary:
        count      mean       std       min       25%       50%       75%  \
Actual
0       212.0  0.074462  0.067278  0.000000  0.025969  0.053703  0.105258
1         8.0  0.051551  0.044184  0.012083  0.016395  0.037445  0.073932

             max
Actual
0       0.440030
1       0.137517

Threshold: 0.5
Confusion Matrix:
[[212   0]
 [  8   0]]

Threshold: 0.2
Confusion Matrix:
[[200  12]
 [  8   0]]

Threshold: 0.1
Confusion Matrix:
[[156  56]
 [  7   1]]

Threshold: 0.05
Confusion Matrix:
[[102 110]
 [  5   3]]

Threshold: 0.02
Confusion Matrix:
[[ 41 171]
 [  3   5]]

Threshold: 0.01
Confusion Matrix:
[[ 19 193]
 [  0   8]]
```

```
import pandas as pd
import numpy as np

# Combine training features and labels
train_analysis = X_train.copy()
train_analysis["Fraud_Label"] = np.asarray(y_train)

# Numerical feature comparison
numeric_cols = train_analysis.select_dtypes(
    include=["int64", "float64"]
).columns.drop("Fraud_Label")

print("NUMERICAL FEATURE COMPARISON")

print(
    train_analysis.groupby("Fraud_Label")[numeric_cols]
    .mean()
    .T.round(2)
)
```

```
NUMERICAL FEATURE COMPARISON
Fraud_Label                          0        1
Transaction_Amount_INR          634.82   572.92
Transaction_Hour                 11.44    12.52
Transactions_Last_Hour            1.69     1.63
Account_Age_Days               1267.86  1300.97
Fraud_Risk_Signal                50.79    56.48
Failed_OTP_Attempts               0.28     0.35
Previous_Disputes                 0.20     0.26
```

```
Minutes_Since_Last_Transaction    90.97   105.00
Hour_Sin                           0.01    -0.24
Hour_Cos                           0.00     0.08
High_Amount                        0.05     0.06
```

```
categorical_cols = [
    "Payment_Method",
    "Transaction_Channel",
    "Device_Status",
    "Location_Type"
]

for col in categorical_cols:
    print("\nFeature:", col)

    print(
        pd.crosstab(
            train_analysis[col],
            train_analysis["Fraud_Label"],
            normalize="columns"
        ).round(3)
    )
```

```
Feature: Payment_Method
Fraud_Label         0      1
Payment_Method
Card            0.251  0.167
NetBanking      0.082  0.100
UPI             0.542  0.600
Wallet          0.124  0.133

Feature: Transaction_Channel
Fraud_Label              0      1
Transaction_Channel
Mobile App           0.336  0.290
POS                  0.344  0.355
Web                  0.320  0.355

Feature: Device_Status
Fraud_Label        0      1
Device_Status
Known          0.786  0.821
New            0.214  0.179

Feature: Location_Type
Fraud_Label        0      1
Location_Type
Domestic       0.895  0.839
International  0.105  0.161
```

```
print("NUMERICAL FEATURE COMPARISON")

print(
    train_analysis.groupby("Fraud_Label")[numeric_cols]
    .mean()
    .T.round(2)
)
```

```
NUMERICAL FEATURE COMPARISON
Fraud_Label                           0        1
Transaction_Amount_INR           634.82   572.92
Transaction_Hour                  11.44    12.52
Transactions_Last_Hour             1.69     1.63
Account_Age_Days                1267.86  1300.97
Fraud_Risk_Signal                 50.79    56.48
Failed_OTP_Attempts                0.28     0.35
Previous_Disputes                  0.20     0.26
Minutes_Since_Last_Transaction    90.97   105.00
Hour_Sin                           0.01    -0.24
Hour_Cos                           0.00     0.08
High_Amount                        0.05     0.06
```

```
print("Reference dataset:")
print(reference["Fraud_Label"].value_counts())

print("\nCurrent dataset:")
print(current["Fraud_Label"].value_counts())
```

```
Reference dataset:
Fraud_Label
Legitimate    1061
Fraudulent      39
Name: count, dtype: int64
```

```
Current dataset:
Fraud_Label
Legitimate    337
Fraudulent     13
Name: count, dtype: int64
```

```
print(
    train_analysis.groupby("Fraud_Label")[numeric_cols]
    .mean()
    .T.round(2)
)
```

```
Fraud_Label                          0        1
Transaction_Amount_INR          634.82   572.92
Transaction_Hour                 11.44    12.52
Transactions_Last_Hour            1.69     1.63
Account_Age_Days               1267.86  1300.97
Fraud_Risk_Signal                50.79    56.48
Failed_OTP_Attempts               0.28     0.35
Previous_Disputes                 0.20     0.26
Minutes_Since_Last_Transaction   90.97   105.00
Hour_Sin                          0.01    -0.24
Hour_Cos                          0.00     0.08
High_Amount                       0.05     0.06
```

```
!pip install -q xgboost
```

```
from xgboost import XGBClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# Create XGBoost model
xgb_model = Pipeline(steps=[
    ("preprocessor", model.named_steps["preprocessor"]),
    ("classifier", XGBClassifier(
        n_estimators=300,
        max_depth=3,
        learning_rate=0.03,
        scale_pos_weight=849/31,
        eval_metric="logloss",
        random_state=42
    ))
])

# Train model
xgb_model.fit(X_train, y_train)

# Predict
xgb_predictions = xgb_model.predict(X_test)

# Evaluate
print("XGBOOST MODEL RESULTS")

print("Accuracy:", accuracy_score(y_test, xgb_predictions))
print("Precision:", precision_score(y_test, xgb_predictions, zero_division=0))
print("Recall:", recall_score(y_test, xgb_predictions, zero_division=0))
print("F1 Score:", f1_score(y_test, xgb_predictions, zero_division=0))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, xgb_predictions))

print("\nClassification Report:")
print(classification_report(
    y_test,
    xgb_predictions,
    target_names=["Legitimate", "Fraudulent"],
    zero_division=0
))
```

```
XGBOOST MODEL RESULTS
Accuracy: 0.9090909090909091
Precision: 0.0
Recall: 0.0
F1 Score: 0.0

Confusion Matrix:
```

```
[[200  12]
 [  8   0]]

Classification Report:
              precision    recall  f1-score   support

  Legitimate       0.96      0.94      0.95       212
  Fraudulent       0.00      0.00      0.00         8

    accuracy                           0.91       220
   macro avg       0.48      0.47      0.48       220
weighted avg       0.93      0.91      0.92       220
```

```
print("Fraud risk signal by class:")

print(
    train_analysis.groupby("Fraud_Label")["Fraud_Risk_Signal"]
    .describe()
)

print("\nFraud risk signal correlation with label:")

print(
    train_analysis["Fraud_Risk_Signal"]
    .corr(train_analysis["Fraud_Label"])
)
```

```
Fraud risk signal by class:
             count       mean        std  min   25%   50%   75%    max
Fraud_Label
0            832.0  50.793269  29.442645  0.0  26.0  51.0  77.0  100.0
1             29.0  56.482759  30.591339  0.0  33.0  58.0  80.0  100.0

Fraud risk signal correlation with label:
0.03483637342635011
```

```
print(
    train_analysis.groupby("Fraud_Label")[
        "Fraud_Risk_Signal"
    ].describe()
)

print("\nCorrelation with fraud label:")

print(
    train_analysis["Fraud_Risk_Signal"]
    .corr(train_analysis["Fraud_Label"])
)
```

```
             count       mean        std  min   25%   50%   75%    max
Fraud_Label
0            832.0  50.793269  29.442645  0.0  26.0  51.0  77.0  100.0
1             29.0  56.482759  30.591339  0.0  33.0  58.0  80.0  100.0

Correlation with fraud label:
0.03483637342635011
```

```
# Check fraud and legitimate transaction counts
print("Fraud label distribution:")
print(train_analysis["Fraud_Label"].value_counts())

print("\nFraud percentage:")
print(train_analysis["Fraud_Label"].value_counts(normalize=True) * 100)
```

```
Fraud label distribution:
Fraud_Label
0    849
1     31
Name: count, dtype: int64

Fraud percentage:
Fraud_Label
0    96.477273
1     3.522727
Name: proportion, dtype: float64
```

```
# Compare numerical features for each fraud class

numeric_columns = [
    "Transaction_Amount_INR",
    "Transaction_Hour",
    "Transactions_Last_Hour",
    "Account_Age_Days",
```

```
    "Fraud_Risk_Signal",
    "Failed_OTP_Attempts",
    "Previous_Disputes",
    "Minutes_Since_Last_Transaction"
]

print(
    train_analysis.groupby("Fraud_Label")[numeric_columns]
    .mean()
    .round(2)
)
```

```
             Transaction_Amount_INR  Transaction_Hour  Transactions_Last_Hour  \
Fraud_Label
0                            634.82             11.44                    1.69
1                            572.92             12.52                    1.63

             Account_Age_Days  Fraud_Risk_Signal  Failed_OTP_Attempts  \
Fraud_Label
0                     1267.86              50.79                 0.28
1                     1300.97              56.48                 0.35

             Previous_Disputes  Minutes_Since_Last_Transaction
Fraud_Label
0                         0.20                          90.97
1                         0.26                         105.00
```

```
import matplotlib.pyplot as plt
import seaborn as sns

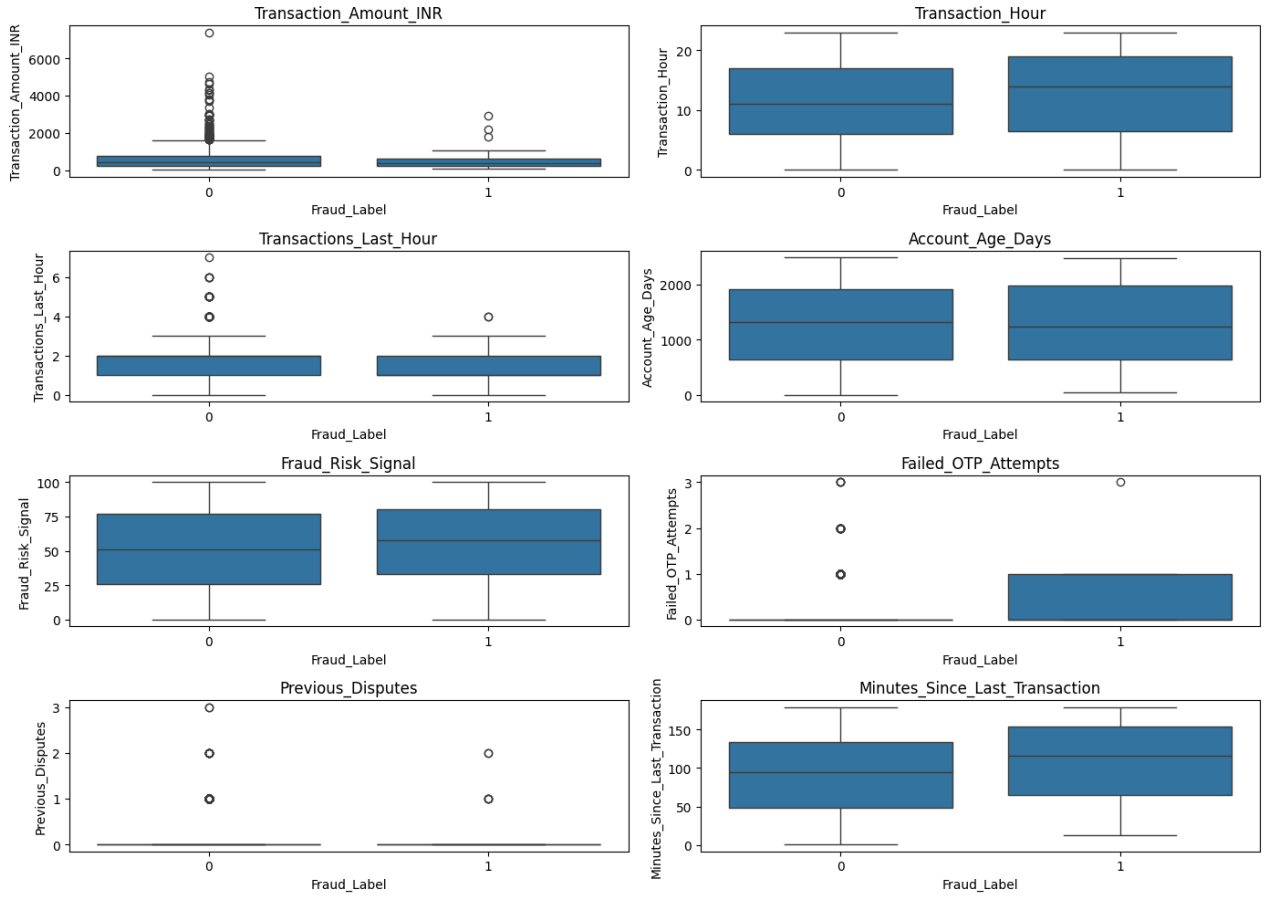
plt.figure(figsize=(14, 10))

for i, col in enumerate(numeric_columns, 1):
    plt.subplot(4, 2, i)

    sns.boxplot(
        data=train_analysis,
        x="Fraud_Label",
        y=col
    )

    plt.title(col)

plt.tight_layout()
plt.show()
```




```
import pandas as pd
import matplotlib.pyplot as plt

categorical_columns = [
    "Payment_Method",
    "Transaction_Channel",
    "Device_Status",
    "Location_Type"
]

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

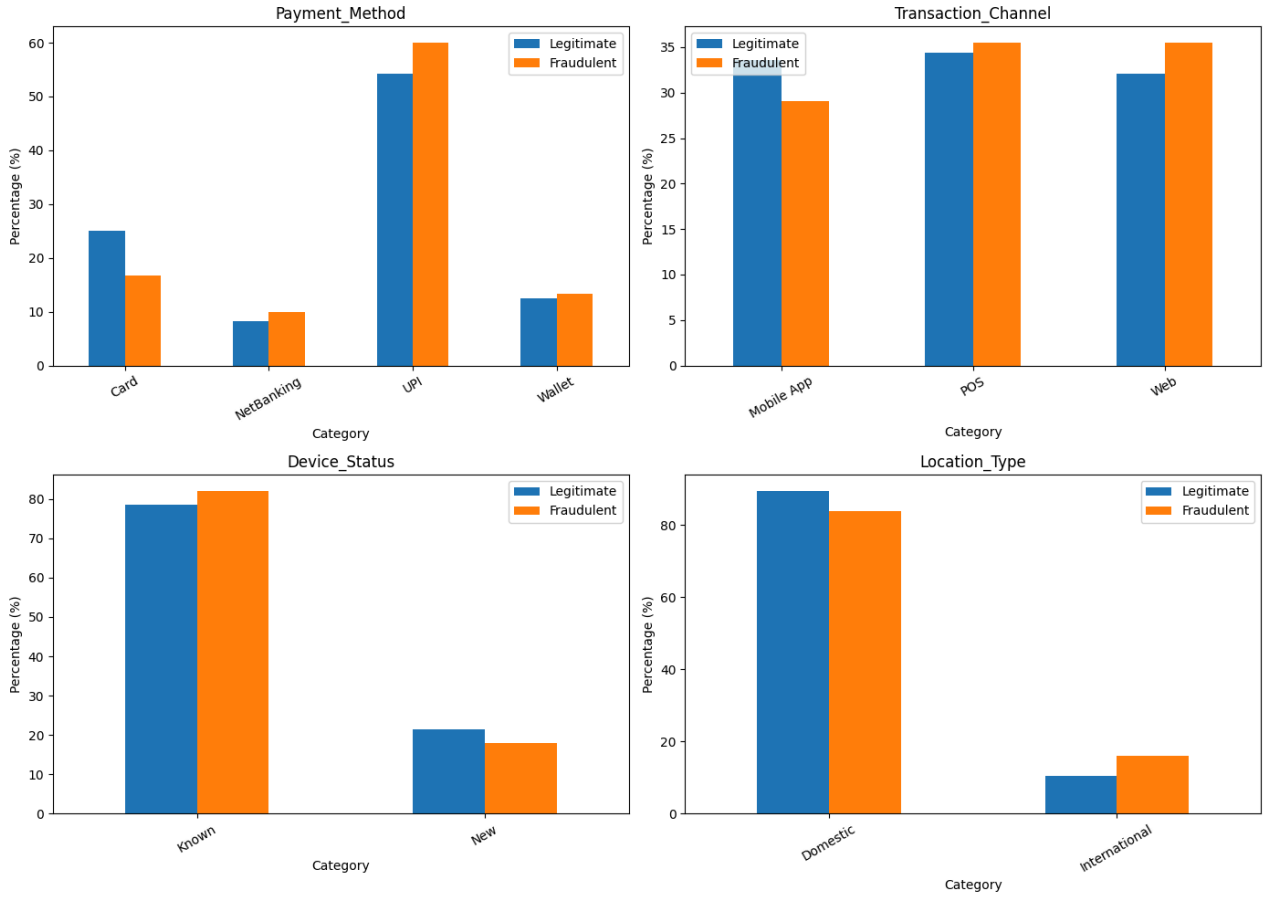
for col, ax in zip(categorical_columns, axes.flatten()):

    # Percentage distribution within each fraud class
    table = pd.crosstab(
        train_analysis[col],
        train_analysis["Fraud_Label"],
        normalize="columns"
    ) * 100

    table.plot(kind="bar", ax=ax)

    ax.set_title(col)
    ax.set_ylabel("Percentage (%)")
    ax.set_xlabel("Category")
    ax.legend(["Legitimate", "Fraudulent"])
    ax.tick_params(axis="x", rotation=30)

plt.tight_layout()
plt.show()
```




```
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score,
    average_precision_score,
    balanced_accuracy_score
)

# Predictions
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Evaluation
print("Classification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=["Legitimate", "Fraudulent"],
    zero_division=0
))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("ROC-AUC Score:", roc_auc_score(y_test, y_prob))
print("PR-AUC Score:", average_precision_score(y_test, y_prob))
print("Balanced Accuracy:", balanced_accuracy_score(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

  Legitimate       0.96      1.00      0.98       212
  Fraudulent       0.00      0.00      0.00         8

    accuracy                           0.96       220
   macro avg       0.48      0.50      0.49       220
weighted avg       0.93      0.96      0.95       220

Confusion Matrix:
[[212   0]
 [  8   0]]
ROC-AUC Score: 0.47788915094339623
```

```
PR-AUC Score: 0.03945943695708026
Balanced Accuracy: 0.5
```

```
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1",
    "roc_auc": "roc_auc"
}

cv_results = cross_validate(
    model,
    X_train,
    y_train,
    cv=cv,
    scoring=scoring,
    return_train_score=False
)

for metric in scoring:
    scores = cv_results["test_" + metric]
    print(f"{metric}: {scores.mean():.4f} (+/- {scores.std():.4f})")
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this beh

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this beh

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this beh

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this beh

accuracy: 0.9648 (+/- 0.0023)
precision: 0.0000 (+/- 0.0000)
recall: 0.0000 (+/- 0.0000)
f1: 0.0000 (+/- 0.0000)
roc_auc: 0.5659 (+/- 0.1174)
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this beh
```

```
import numpy as np
import matplotlib.pyplot as plt

# Fraud probabilities from the test dataset
fraud_probabilities = model.predict_proba(X_test)[:, 1]

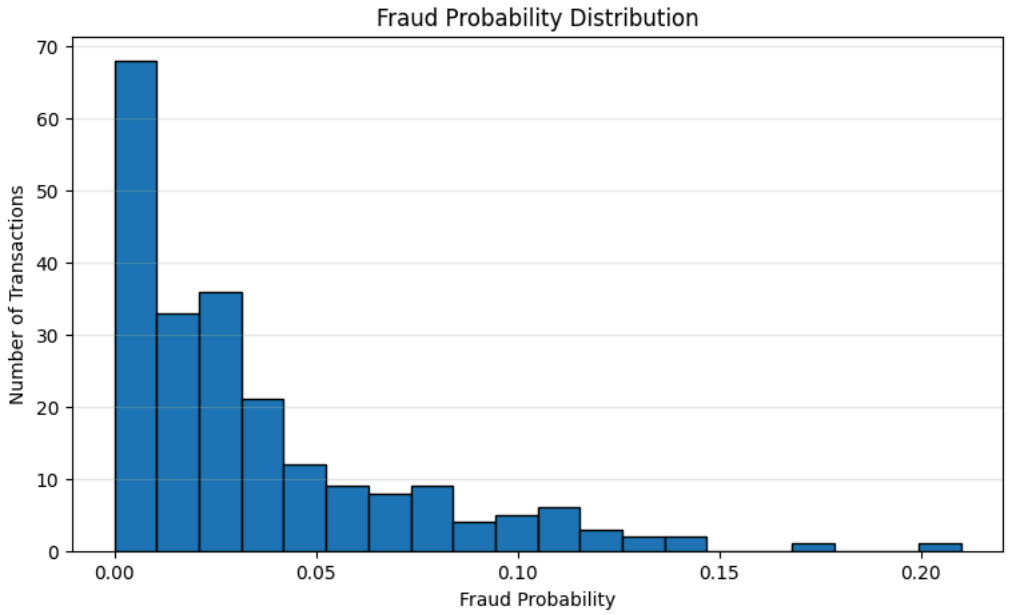
plt.figure(figsize=(9, 5))

plt.hist(fraud_probabilities, bins=20, edgecolor="black")

plt.xlabel("Fraud Probability")
plt.ylabel("Number of Transactions")
plt.title("Fraud Probability Distribution")
plt.grid(axis="y", alpha=0.3)

plt.show()

print("Minimum probability:", np.min(fraud_probabilities))
print("Maximum probability:", np.max(fraud_probabilities))
print("Average probability:", np.mean(fraud_probabilities))
```




```
Minimum probability: 0.0
Maximum probability: 0.21
Average probability: 0.035090909090909096
```

```
import pandas as pd

probability_analysis = pd.DataFrame({
    "Actual_Label": y_test.values,
    "Fraud_Probability": model.predict_proba(X_test)[:, 1]
})

print(
    probability_analysis.groupby("Actual_Label")[
        "Fraud_Probability"
    ].describe().round(4)
)
```

```
              count    mean     std  min    25%    50%   75%   max
Actual_Label
0             212.0  0.0353  0.0364  0.0  0.010  0.025  0.05  0.21
1               8.0  0.0300  0.0274  0.0  0.005  0.025  0.05  0.07
```

```
import matplotlib.pyplot as plt
import seaborn as sns

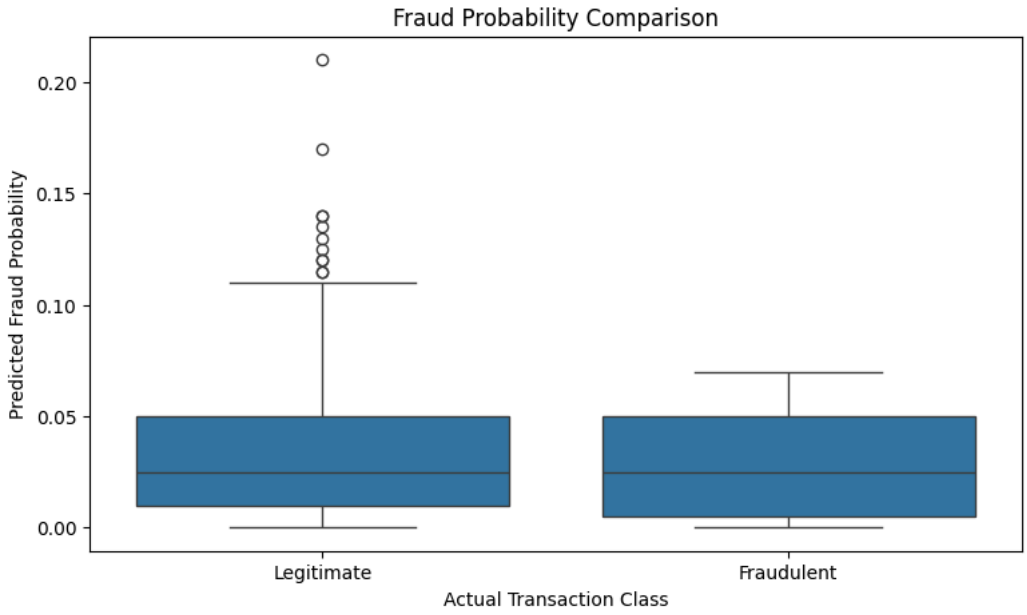
plt.figure(figsize=(9, 5))

sns.boxplot(
    data=probability_analysis,
    x="Actual_Label",
    y="Fraud_Probability"
)

plt.xticks(
    [0, 1],
    ["Legitimate", "Fraudulent"]
)

plt.xlabel("Actual Transaction Class")
plt.ylabel("Predicted Fraud Probability")
plt.title("Fraud Probability Comparison")

plt.show()
```




```
import joblib

# Save the original Random Forest pipeline
joblib.dump(model, "/content/fraud_model_final.pkl")

print("Final baseline model saved successfully!")
```

```
Final baseline model saved successfully!
```

```
loaded_model = joblib.load("/content/fraud_model_final.pkl")

print("Model loaded successfully!")

predictions = loaded_model.predict(X_test)

print("Total test transactions:", len(predictions))
print("Legitimate predictions:", (predictions == 0).sum())
print("Fraudulent predictions:", (predictions == 1).sum())
```

```
Model loaded successfully!
Total test transactions: 220
Legitimate predictions: 220
Fraudulent predictions: 0
```

```
import requests

response = requests.get("http://127.0.0.1:8000/metrics")

print("Status Code:", response.status_code)
print("\nPrometheus Metrics:\n")
print(response.text[:3000])
```

```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 1122.0
python_gc_objects_collected_total{generation="1"} 35.0
python_gc_objects_collected_total{generation="2"} 48.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 138.0
python_gc_collections_total{generation="1"} 12.0
python_gc_collections_total{generation="2"} 1.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="13",patchlevel="18",version="3.13.18"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 8.4310016e+08
```

```
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.79397703639e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 11.14
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 17.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP transaction_request_count_total Total transaction requests
# TYPE transaction_request_count_total counter
transaction_request_count_total 1.0
# HELP transaction_request_count_created Total transaction requests
# TYPE transaction_request_count_created gauge
transaction_request_count_created 1.793977040673813e+09
# HELP fraud_prediction_count_total Total predictions
# TYPE fraud_prediction_count_total counter
fraud_prediction_count_total 1.0
# HELP fraud_prediction_count_created Total predictions
# TYPE fraud_prediction_count_created gauge
fraud_prediction_count_created 1.7939770406738687e+09
# HELP application_error_count_total Total errors
# TYPE application_error_count_total counter
application_error_count_total 0.0
# HELP application_error_count_created Total errors
# TYPE application_error_count_created gauge
application_error_count_created 1.7939770406738918e+09
# HELP transaction_response_time_seconds Transaction processing time
# TYPE transaction_response_time_seconds histogram
transaction_response_time_seconds_bucket{le="0.005"} 0.0
transaction_response_time_seconds_b
```

```
%%writefile /content/prometheus.yml
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: "fraud-detection-api"
    metrics_path: "/metrics"
    static_configs:
      - targets: ["127.0.0.1:8000"]
```

```
Writing /content/prometheus.yml
```

```
!apt-get update -qq
!apt-get install -y prometheus
```

```
/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link
```

```
!nohup prometheus \
  --config.file=/content/prometheus.yml \
  --web.listen-address=0.0.0.0:9090 \
  > /content/prometheus.log 2>&1 &
```

```
import requests
import time

time.sleep(3)

response = requests.get("http://127.0.0.1:9090/-/healthy")

print("Prometheus status:", response.status_code)
print(response.text)
```

```
Prometheus status: 200
Prometheus Server is Healthy.
```

```
import requests

url = "http://127.0.0.1:9090/api/v1/targets"

response = requests.get(url)
data = response.json()

for target in data["data"]["activeTargets"]:
    print("Job:", target["labels"]["job"])
    print("Health:", target["health"])
    print("Last Error:", target["lastError"])
    print("Scrape URL:", target["scrapeUrl"])
```

```
Job: fraud-detection-api
Health: up
Last Error:
Scrape URL: http://127.0.0.1:8000/metrics
```

```
!apt-get install -y apt-transport-https software-properties-common wget
!wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor -o /usr/share/keyrings/grafana.gpg

!echo "deb [signed-by=/usr/share/keyrings/grafana.gpg] https://apt.grafana.com stable main" | tee /etc/apt/sources.list.d/gr

!apt-get update -qq
!apt-get install -y grafana
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
software-properties-common is already the newest version (0.99.49.4).
wget is already the newest version (1.21.4-1ubuntu4.5).
The following additional packages will be installed:
  apt apt-utils libapt-pkg-dev libapt-pkg6.0t64
Suggested packages:
  apt-doc aptitude | synaptic | wajig powermgmt-base
The following NEW packages will be installed:
  apt-transport-https
The following packages will be upgraded:
  apt apt-utils libapt-pkg-dev libapt-pkg6.0t64
4 upgraded, 1 newly installed, 0 to remove and 27 not upgraded.
Need to get 2,842 kB of archives.
After this operation, 395 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libapt-pkg-dev amd64 2.8.4ubuntu0.1 [119 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libapt-pkg6.0t64 amd64 2.8.4ubuntu0.1 [985 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 apt amd64 2.8.4ubuntu0.1 [1,397 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 apt-utils amd64 2.8.4ubuntu0.1 [315 kB]
```

```
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 apt-transport-https all 2.8.4ubuntu0.1 [26.2 kB]
Fetched 2,842 kB in 0s (18.3 MB/s)
(Reading database ... 124551 files and directories currently installed.)
Preparing to unpack .../libapt-pkg-dev_2.8.4ubuntu0.1_amd64.deb ...
Unpacking libapt-pkg-dev:amd64 (2.8.4ubuntu0.1) over (2.8.3) ...
Preparing to unpack .../libapt-pkg6.0t64_2.8.4ubuntu0.1_amd64.deb ...
Unpacking libapt-pkg6.0t64:amd64 (2.8.4ubuntu0.1) over (2.8.3) ...
Setting up libapt-pkg6.0t64:amd64 (2.8.4ubuntu0.1) ...
(Reading database ... 124551 files and directories currently installed.)
Preparing to unpack .../apt_2.8.4ubuntu0.1_amd64.deb ...
Unpacking apt (2.8.4ubuntu0.1) over (2.8.3) ...
Setting up apt (2.8.4ubuntu0.1) ...
(Reading database ... 124551 files and directories currently installed.)
Preparing to unpack .../apt-utils_2.8.4ubuntu0.1_amd64.deb ...
Unpacking apt-utils (2.8.4ubuntu0.1) over (2.8.3) ...
Selecting previously unselected package apt-transport-https.
Preparing to unpack .../apt-transport-https_2.8.4ubuntu0.1_all.deb ...
Unpacking apt-transport-https (2.8.4ubuntu0.1) ...
Setting up apt-utils (2.8.4ubuntu0.1) ...
Setting up apt-transport-https (2.8.4ubuntu0.1) ...
Setting up libapt-pkg-dev:amd64 (2.8.4ubuntu0.1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic link
```

```
!nohup grafana-server \
  --homepath /usr/share/grafana \
  --config /etc/grafana/grafana.ini \
  > /content/grafana.log 2>&1 &
```

```
!grafana-server -v
!sudo service grafana-server status
!tail -50 /content/grafana.log
```

```
Deprecation warning: 'grafana-server' is deprecated and will be removed in a future release. Use the 'grafana server' subcom
Version 13.3.0 (commit: 6f8ce3c693b493043ebcd4787566d2b6b118b0f0, branch: release-13.3.0#patched)
grafana-server: unrecognized service
logger=search-manifest-watcher t=2026-11-06T16:50:23.9970159Z level=warn msg="manifest watcher: source listed zero manifests
logger=search-manifest-watcher t=2026-11-06T16:50:23.99710826Z level=warn msg="manifest watcher poll cycle: zero manifests,
logger=installer.fs t=2026-11-06T16:50:25.808843222Z level=info msg="Downloaded and extracted prometheus v13.2.5 zip success
logger=plugin.prometheus t=2026-11-06T16:50:25.81316011Z level=info msg="plugin process exited" plugin=/usr/share/grafana/da
logger=plugin.installer t=2026-11-06T16:50:26.275075504Z level=info msg="Installed plugin update" pluginId=prometheus path=/
logger=plugins.registration t=2026-11-06T16:50:26.635678403Z level=info msg="Plugin registered" pluginId=prometheus pluginVe
logger=plugin.backgroundinstaller t=2026-11-06T16:50:26.635781244Z level=info msg="Plugin successfully installed" pluginId=p
logger=plugin.backgroundinstaller t=2026-11-06T16:50:26.662348444Z level=info msg="Installing plugin" pluginId=influxdb vers
logger=plugin.installer t=2026-11-06T16:50:26.681334431Z level=info msg="Updating plugin" pluginId=influxdb from=13.1.7 to=1
logger=installer.fs t=2026-11-06T16:50:28.334090537Z level=info msg="Downloaded and extracted influxdb v13.1.10 zip successf
logger=plugin.influxdb t=2026-11-06T16:50:28.337246608Z level=info msg="plugin process exited" plugin=/usr/share/grafana/dat
logger=plugin.installer t=2026-11-06T16:50:28.343818489Z level=info msg="Installed plugin update" pluginId=influxdb path=/us
logger=plugins.registration t=2026-11-06T16:50:28.546289147Z level=info msg="Plugin registered" pluginId=influxdb pluginVers
logger=plugin.backgroundinstaller t=2026-11-06T16:50:28.546410134Z level=info msg="Plugin successfully installed" pluginId=i
logger=plugin.backgroundinstaller t=2026-11-06T16:50:28.546453636Z level=info msg="Installing plugin" pluginId=grafana-explo
logger=plugin.installer t=2026-11-06T16:50:29.627309689Z level=info msg="Installing plugin" pluginId=grafana-exploretraces-a
logger=installer.fs t=2026-11-06T16:50:29.744350588Z level=info msg="Downloaded and extracted grafana-exploretraces-app v3.2
logger=plugins.registration t=2026-11-06T16:50:29.807371185Z level=info msg="Plugin registered" pluginId=grafana-exploretrac
logger=plugin.backgroundinstaller t=2026-11-06T16:50:29.80744629Z level=info msg="Plugin successfully installed" pluginId=gr
logger=plugin.backgroundinstaller t=2026-11-06T16:50:29.839485682Z level=info msg="Installing plugin" pluginId=cloudwatch ve
logger=plugin.installer t=2026-11-06T16:50:29.875050503Z level=info msg="Updating plugin" pluginId=cloudwatch from=12.11.1 t
logger=installer.fs t=2026-11-06T16:50:32.85165527Z level=info msg="Downloaded and extracted cloudwatch v12.11.2 zip success
logger=plugin.cloudwatch t=2026-11-06T16:50:32.854590779Z level=info msg="plugin process exited" plugin=/usr/share/grafana/c
logger=plugin.installer t=2026-11-06T16:50:33.363224095Z level=info msg="Installed plugin update" pluginId=cloudwatch path=/
logger=plugins.registration t=2026-11-06T16:50:34.152371009Z level=info msg="Plugin registered" pluginId=cloudwatch pluginVe
logger=plugin.backgroundinstaller t=2026-11-06T16:50:34.152470888Z level=info msg="Plugin successfully installed" pluginId=c
logger=plugin.backgroundinstaller t=2026-11-06T16:50:34.152514497Z level=info msg="Installing plugin" pluginId=grafana-advis
logger=plugin.installer t=2026-11-06T16:50:34.959924319Z level=info msg="Installing plugin" pluginId=grafana-advisor-app ver
logger=installer.fs t=2026-11-06T16:50:34.987477521Z level=info msg="Downloaded and extracted grafana-advisor-app v1.1.2 zip
logger=plugins.registration t=2026-11-06T16:50:35.030836088Z level=info msg="Plugin registered" pluginId=grafana-advisor-app
logger=plugin.backgroundinstaller t=2026-11-06T16:50:35.030914237Z level=info msg="Plugin successfully installed" pluginId=g
logger=plugin.backgroundinstaller t=2026-11-06T16:50:35.056137824Z level=info msg="Installing plugin" pluginId=tempo version
logger=plugin.installer t=2026-11-06T16:50:35.095977624Z level=info msg="Updating plugin" pluginId=tempo from=13.3.0 to=13.4
logger=installer.fs t=2026-11-06T16:50:37.523775193Z level=info msg="Downloaded and extracted tempo v13.4.2 zip successfully
logger=plugin.tempo t=2026-11-06T16:50:37.527045211Z level=info msg="plugin process exited" plugin=/usr/share/grafana/data/p
logger=plugin.installer t=2026-11-06T16:50:37.53538029Z level=info msg="Installed plugin update" pluginId=tempo path=/usr/sh
logger=plugins.registration t=2026-11-06T16:50:38.129173052Z level=info msg="Plugin registered" pluginId=tempo pluginVersion
logger=plugin.backgroundinstaller t=2026-11-06T16:50:38.12930071Z level=info msg="Plugin successfully installed" pluginId=te
```

```
logger=plugin.backgroundinstaller t=2026-11-06T16:50:38.175711734Z level=info msg="Installing plugin" pluginId=zipkin versio
logger=plugin.installer t=2026-11-06T16:50:38.198414253Z level=info msg="Updating plugin" pluginId=zipkin from=12.4.11 to=12
logger=installer.fs t=2026-11-06T16:50:40.559650197Z level=info msg="Downloaded and extracted zipkin v12.4.12 zip successful
logger=plugin.zipkin t=2026-11-06T16:50:40.564487375Z level=info msg="plugin process exited" plugin=/usr/share/grafana/data/
logger=plugin.installer t=2026-11-06T16:50:40.676801016Z level=info msg="Installed plugin update" pluginId=zipkin path=/usr/
logger=plugins.registration t=2026-11-06T16:50:40.978989173Z level=info msg="Plugin registered" pluginId=zipkin pluginVersio
logger=plugin.backgroundinstaller t=2026-11-06T16:50:40.979101665Z level=info msg="Plugin successfully installed" pluginId=z
logger=plugin.backgroundinstaller t=2026-11-06T16:50:40.979185067Z level=info msg="Installing plugin" pluginId=grafana-pyros
logger=plugin.installer t=2026-11-06T16:50:42.17226234Z level=info msg="Installing plugin" pluginId=grafana-pyroscope-app ve
logger=installer.fs t=2026-11-06T16:50:42.583878071Z level=info msg="Downloaded and extracted grafana-pyroscope-app v3.1.0 z
logger=plugins.registration t=2026-11-06T16:50:42.979822398Z level=info msg="Plugin registered" pluginId=grafana-pyroscope-a
logger=plugin.backgroundinstaller t=2026-11-06T16:50:42.979908815Z level=info msg="Plugin successfully installed" pluginId=g
```

```
!sudo service grafana-server start
```

```
grafana-server: unrecognized service
```

```
!sudo service grafana-server status
```

```
grafana-server: unrecognized service
```

```
import subprocess
import requests
import time

# 1. Start Grafana in the background manually since systemd/service is unavailable
print("Starting Grafana server in the background...")
subprocess.Popen(
    ["grafana-server", "--homepath", "/usr/share/grafana", "--config", "/etc/grafana/grafana.ini"],
    stdout=open("/content/grafana_manual.log", "w"),
    stderr=subprocess.STDOUT
)

# 2. Wait for Grafana to initialize
time.sleep(5)

# 3. Test the health endpoint
try:
    response = requests.get("http://127.0.0.1:3000/api/health", timeout=5)
    print("Status:", response.status_code)
    print("Response:", response.json())
except Exception as e:
    print("Failed to connect to Grafana:", str(e))
    print("\n--- Tail of Grafana Logs ---")
    with open("/content/grafana_manual.log", "r") as f:
        print("".join(f.readlines()[-30:]))
```

```
Starting Grafana server in the background...
Status: 200
Response: {'database': 'ok', 'version': '13.3.0', 'commit': '6f8ce3c693b493043ebcd4787566d2b6b118b0f0'}
```

```
import requests

url = "http://127.0.0.1:3000/api/datasources"

payload = {
    "name": "Prometheus",
    "type": "prometheus",
    "access": "proxy",
    "url": "http://127.0.0.1:9090",
    "isDefault": True
}

response = requests.post(
    url,
    auth=("admin", "admin"),
    json=payload
)

print("Status:", response.status_code)
print(response.text)
```

```
Status: 200
{"datasource":{"id":1,"uid":"ag3i1oqgjedq8f","orgId":1,"name":"Prometheus","type":"prometheus","typeLogoUrl":"public/plugins
```

```
import requests

url = "http://127.0.0.1:3000/api/datasources"

payload = {
```

```
    "name": "Prometheus",
    "type": "prometheus",
    "access": "proxy",
    "url": "http://127.0.0.1:9090",
    "isDefault": True
}

response = requests.post(
    url,
    auth=("admin", "admin"),
    json=payload
)

print("Status:", response.status_code)
print(response.text)
```

```
Status: 409
{"message":"data source with the same name already exists"}
```

```
response = requests.get(
    "http://127.0.0.1:3000/api/datasources",
    auth=("admin", "admin")
)

print(response.json())
```

```
[{'id': 1, 'uid': 'ag3i1oqgjedq8f', 'orgId': 1, 'name': 'Prometheus', 'type': 'prometheus', 'typeName': 'Prometheus', 'typeL
```

```
import requests

url = "http://127.0.0.1:3000/api/dashboards/db"

dashboard = {
    "dashboard": {
        "id": None,
        "title": "Online Payment Fraud Detection Monitoring",
        "tags": ["fraud-detection", "ml-monitoring"],
        "timezone": "browser",
        "schemaVersion": 39,
        "version": 0,
        "refresh": "5s",
        "panels": [
            {
                "id": 1,
                "title": "Total Transaction Requests",
                "type": "stat",
                "datasource": {"type": "prometheus", "uid": "ag3i1oqgjedq8f"},
                "targets": [{"expr": "transaction_request_count_total", "refId": "A"}],
                "gridPos": {"h": 8, "w": 12, "x": 0, "y": 0}
            },
            {
                "id": 2,
                "title": "Fraud Prediction Count",
                "type": "stat",
                "datasource": {"type": "prometheus", "uid": "ag3i1oqgjedq8f"},
                "targets": [{"expr": "fraud_prediction_count_total", "refId": "A"}],
                "gridPos": {"h": 8, "w": 12, "x": 12, "y": 0}
            },
            {
                "id": 3,
                "title": "Application Errors",
                "type": "stat",
                "datasource": {"type": "prometheus", "uid": "ag3i1oqgjedq8f"},
                "targets": [{"expr": "application_error_count_total", "refId": "A"}],
                "gridPos": {"h": 8, "w": 12, "x": 0, "y": 8}
            },
            {
                "id": 4,
                "title": "Transaction Response Time",
                "type": "timeseries",
                "datasource": {"type": "prometheus", "uid": "ag3i1oqgjedq8f"},
                "targets": [{
                    "expr": "rate(transaction_response_time_seconds_sum[1m]) / clamp_min(rate(transaction_response_time_sec
                    "refId": "A"
                }],
                "gridPos": {"h": 8, "w": 12, "x": 12, "y": 8}
            }
        ]
```